C Programming

Course Title : C programming

Code:

Sub. Teacher: Dinesh Khadka

Course Detail

- Unit 1: Problem Solving With Computer
- Unit 2: C Basic
- Unit 3: Operator and expression
- Unit 4: Input and Output
- Unit 5: Control structure
- Unit 6: Array
- Unit 7: User defined functions
- Unit 8: Pointers
- Unit 9: Structure
- Unit 10: Data file Handling
- Unit 11: Introduction to Graphics

By: Dinesh Khadka

Laboratory Work

- Laboratory work should be done to cover all the topics .
  - TC compiler
  - Dev C++ compiler
- Project work

By: Dinesh Khadka

Evaluation

- Internal Assessment – 20
- Final Exam – 60
- Practical - 20

By: Dinesh Khadka

Books

- Programming in ANSI C – E Balagurusamy
- Programming with C – Byrons S Gotterfried

.

By: Dinesh Khadka

Unit :1

Programming Language

Problem Solving with Computer

- There are many methods and methodologies for developing a program, but each is generally consist of series of defined steps and stages.
- the different stages used in problem solving are:
  - Problem Analysis
  - Program design (algorithm development, drawing flowchart)
  - Coding
  - Debugging and testing
  - Maintenance
  - Documentation

By: Dinesh Khadka

Problem analysis

- First the requirement is identified and defined.
- The programmer and analyst review the specifications and possibly communicate with users to understand the requirement.
- Problem that is identified and defined will be analyzed
  - Define the objective of the program.
  - Determine the outputs
  - Determine the input requirement
  - Determine the processing requirements
  - Perform feasibility analysis

By: Dinesh Khadka

Program design

- Next step is develop the logical design of the program .
- In this phase the problem that is identified and analyzed in previous phase will converted into design frame (gives the detail logic of program development).
- Some of the tools used for program design are:
  - Algorithm, Flowchart, Data flow diagram (DFD), ER diagram etc.
- The output of this phase is the foundation for the coding phase.
- For designing we can use top down or bottom up approach.

By: Dinesh Khadka

Coding

- Also called writing program.
- It is the process of converting the program logic design into computer program using syntax of programming language of concern.
- The program developed using programming language is called source program or source code.
- For coding knowledge of programming language is mandatory.
- We choose one of the language such as : C , C++, JAVA, Python etc. for coding

By: Dinesh Khadka

Debugging and testing

- After coding, source code is converted into object code.
- Then only the program will be executed.
- But before executing debugging and testing should be carried out.
- Debugging is the process of locating and fixing the programming errors that may occurs during compilation, or execution of program.
- Testing is the process of verifying that the developed program fulfills the objectives of development.
- Testing is carried out to find out the errors and detects, if there is any difference between the actual output and desired output.

By: Dinesh Khadka

Maintenance

- Longest phase

By: Dinesh Khadka

Documentation

- User documentation
- Programmer documentation

By: Dinesh Khadka

Program design tools

Algorithm

- A set of rule that define how a particular problem can be solved in a finite number of steps is known as algorithm.
- An algorithm is composed of finite number of steps, each of which may require one or more operations.
- It is step by step logical representation of solution of problem.
- It is programming language independent.
- Once we have an algorithm, it act as basis for coding.

By: Dinesh Khadka

Properties of an algorithm

An algorithm should have the following properties

- **Inputs/outputs:** There must be some inputs from standard set of input(s), an algorithms execution must produce output(s).
- **Definiteness:** Each step in algorithm must be clear and unambiguous.
- **Finiteness:** An algorithm should be finite means if executed it should be finished in finite steps.
- **Correctness:** Correct set of output values must be produced from the each set of inputs.
- **Effectiveness:** An algorithm should be effective means each step must be carried out in finite time.

By: Dinesh Khadka

Examples

Example 1: Algorithm to find area and perimeter of rectangle.

Step 1: START

Step 2: Print "Enter length and breadth of rectangle"

Step 3: Read L and B

Step 4: Area = L × B

Step 4: P = 2 × (L + B)

Step 5: Print A as area and P as perimeter.

Step 6: STOP

By: Dinesh Khadka

Example 2: Algorithm to check whether a number is odd or even.

Step 1: START

Step 2: Display "Enter a number"

Step 3: Read **n**

Step 4: r = n mod 2

Step 5: is r=0 ? If yes goto 6 if no goto 7

Step 6: Display "Even" and goto step 8.

Step 7: Display " Odd"

Step 8: STOP

By: Dinesh Khadka

Algorithm to check whether a number is negative or positive

Step 1: START

Step 2: Print " Enter an number"

Step 3: Read N

Step 4: is N<0? If yes goto 5 if no goto 6

Step 5: Print "Negative" and goto 9

Step 6: is N>0? If yes goto 7 if no goto 8

Step 7: Print "Positive" and goto 9

Step 8: Print "zero"

Step 9: STOP

By: Dinesh Khadka

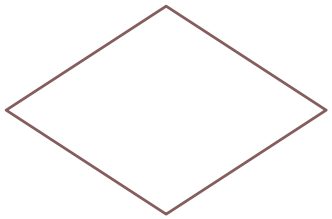
Flowchart

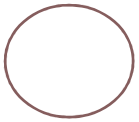
- A flow chart is a pictorial representation of step by step solution of a problem.
- Flow chart not only pictorially depicts the sequence in which instructions are carried out in algorithm but is also used an aid in developing algorithm.
- There are various symbol used in program flowchart, some essential symbol are:-

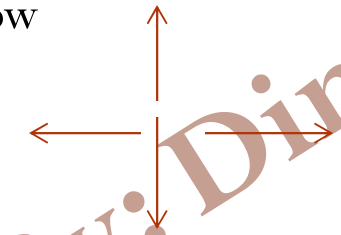
By: Dinesh Khadka

Symbol	Meaning
oval/terminator	**Terminal symbol:** it indicates the beginning and the end of the programs.
Parallelogram	**Input/output symbol:** it indicates input and output operation. It is appear where data is to be read (entered) and where result is to be displayed.
Rectangle	**Processing symbol:** it indicates any processing like calculation and manipulation.

By: Dinesh Khadka

Diamond 	**Decision symbol:** it is used to check the logical conditions. It has one entry point and two exit points which are for true and false.

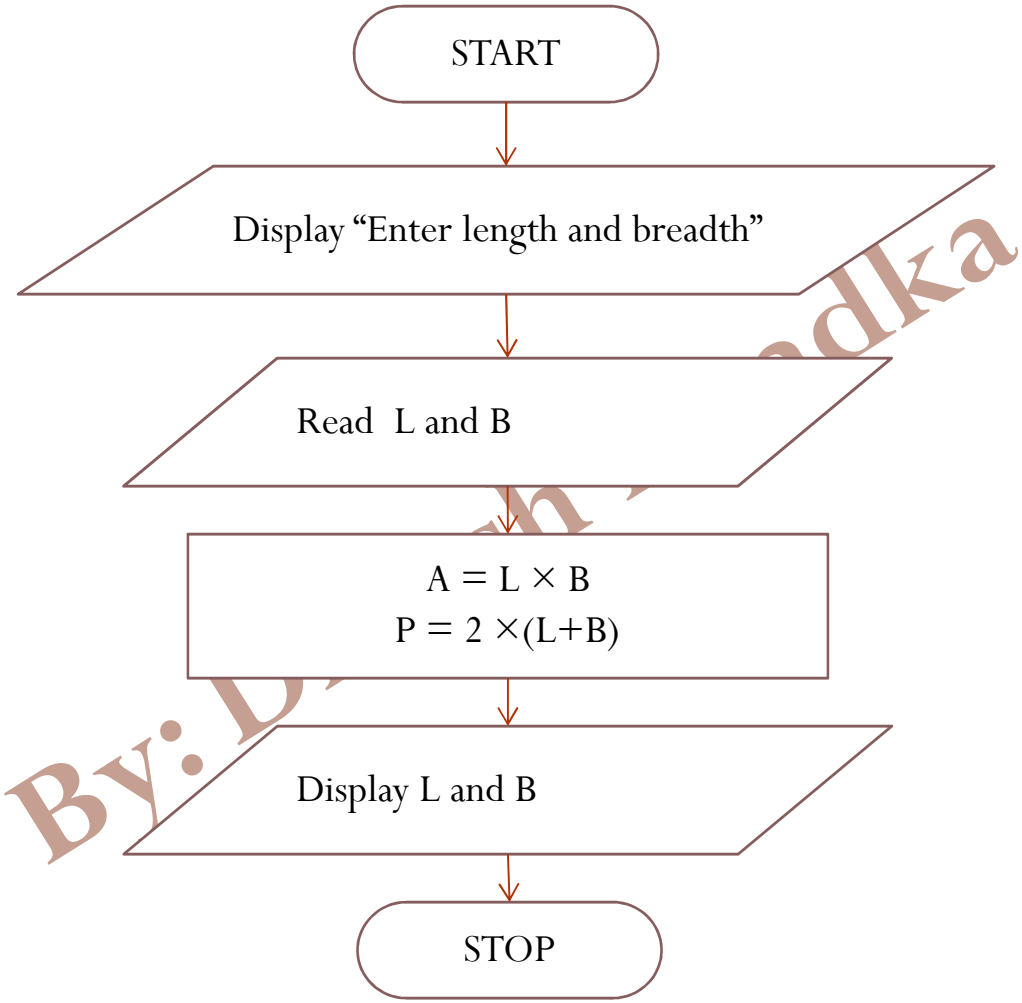
Circle 	**Connector:** it is used to connect the long flowchart that does not fit in the same page. In this case the floe chart is broken into two parts and a arrow is drawn to the connector containing the unique symbol or letter.

Line with arrow 	**Flow line:** it is used to connect the flowchart symbols.

By: Dinesh Khadka

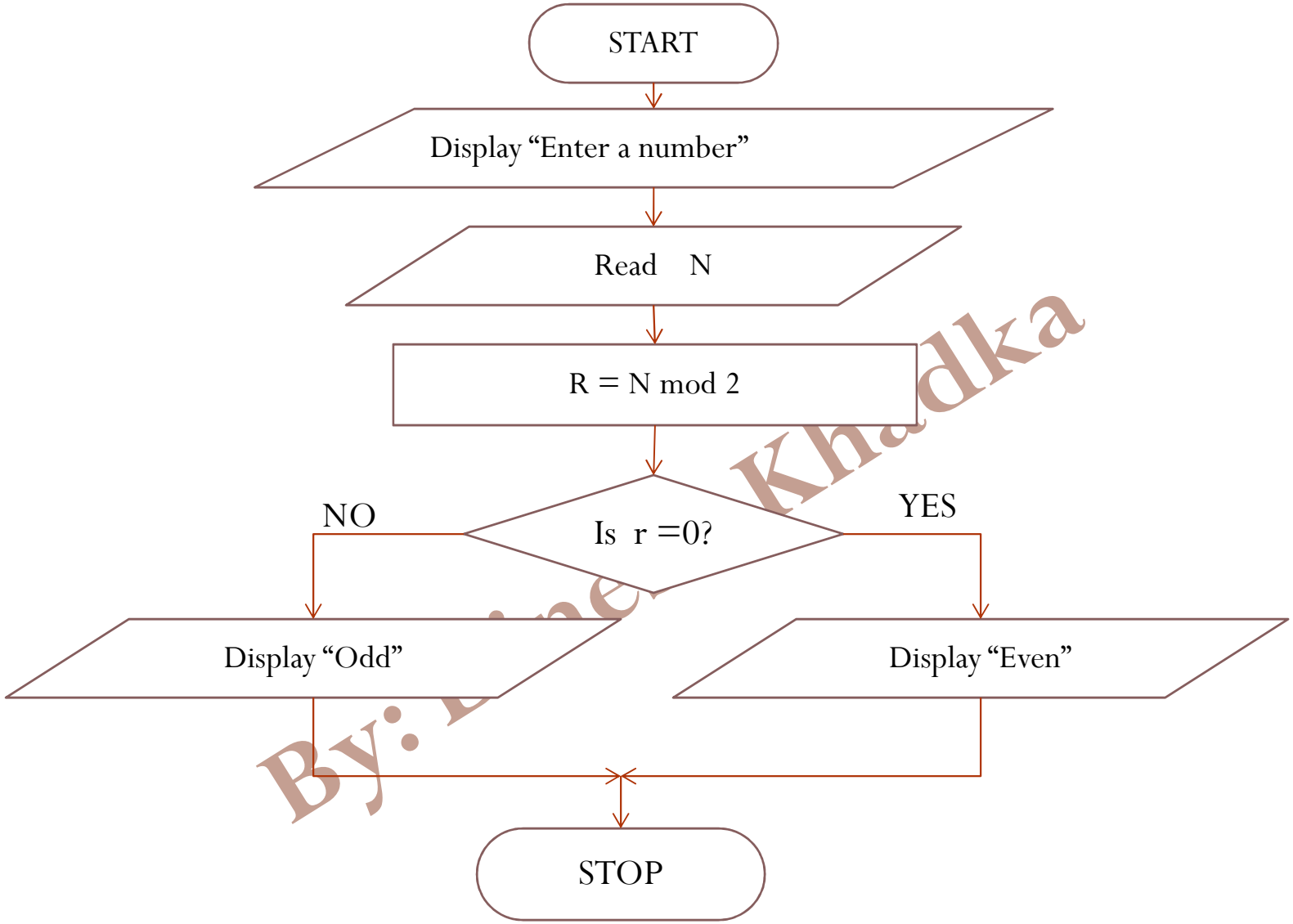
Example

Flowchart to calculate area and perimeter of rectangle



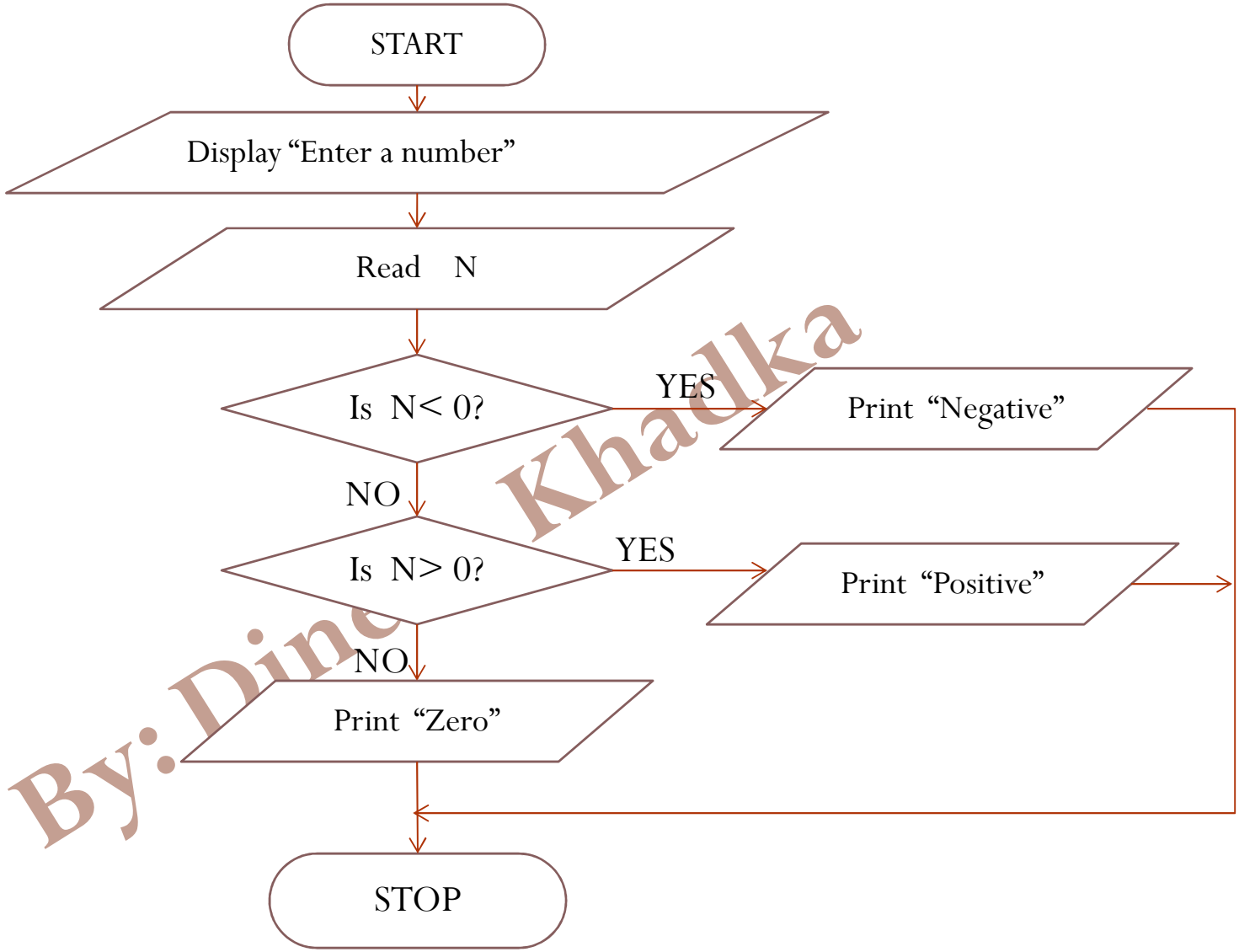
By: Dinesh Khadka

Flowchart to check whether a number is odd or even.



By: Dinesh Khadka

Flowchart to check whether a number is negative or positive



By: Dinesh Khadka

Program errors

- very few programs run correctively for the first time.
- Errors may occurs at different stage of programming.
- Any error in program is called "bug" and is a piece of code that prevents a program from compiling, and running correctly .
- Program debugging is process of finding or discovering and correction of programming errors. That is isolating error from program is called debugging.
- Debugging is basically done to find and fix the following types of programming errors.
  - Syntax error
  - Run-time errors
  - Logical errors
  - Latent errors

By: Dinesh Khadka

Syntax error

- Any violation of rules of the language results in syntax errors.
- Detected with the help of compiler.
- when syntax errors are presents, the compilation fails and is terminated after listing errors and the line numbers in the source program, where error have occurred.
- After viewing error message, programmer can simplify fixed it.
- Example: if we write the following statement to add value of a and b in C language

  Sum = a +b

- Then compiler report the error message, semicolon missing as every C statement should terminate with semicolon. This can be fixed by adding semicolon as

  Sum = a + b ;

By: Dinesh Khadka

Runtime errors

- Occurs during the execution of program.
- Execution of program with runtime errors will terminates abnormally.
- That is program with runtime errors will compiled, run , takes input but do not produce the complete result.
- Runtime errors are due to :
  - Attempt to divide by zero
  - Trying to open a fie that is not created
  - Lack of memory space etc.
- In C run time errors can be handled by using **if statement**. But in C++, JAVA it can be handled by using exception handling features.

By: Dinesh Khadka

Logical errors

- Errors due to poor understanding of the problem, incorrect translation of the algorithm into the program and a lack of clarity of hierarchy of operators are logical errors.
- Compiler can not catch the logical errors.
- Program with logical error will compiled, run ,takes input but produce undesired result.
- To debug the logical errors
  - First we have to run the program, gives the input data.
  - If it gives the undesired result, then we know there is logical error in program.
  - To detect the error, we have to check the entire program line by line or trace it or needs help of expert.
  - Then only it can be fixed by writing appropriate code.
- Example: if we write s = a – b; to add two numbers then the program with the this could wont gives desired result. (a logical error)

By: Dinesh Khadka

Latent errors

- It is hidden error that shows up only when a particular set of data is used.
- For example, consider the following statement:

  R=(x+y)/(p-q);
- An error only occurs when p and q are same.
- An errors of this kind is detected only when all possible combination of test data are used.

By: Dinesh Khadka

Basic of C

Introduction

- It is a popular computer programming language.
- C is the procedural, general purpose, high level language.
- Also called middle level language. (feature of both low level and high level language)
- Used for
  - System programming
    - OS, Compiler
  - Application programming.

By: Dinesh Khadka

History of C

- C programming language was developed in 1972 by Dennis Ritchie at bell laboratories of AT&T(American Telephone & Telegraph), located in U.S.A.
- Dennis Ritchie is known as founder of c language.
- It was developed to be used in UNIX Operating system.
- It inherits many features of previous languages such as B and BPCL.

By: Dinesh Khadka

History of c programming

Language	year	Developed By
ALGOL	1960	International Group
BPCL	1967	Martin Richards
B	1970	Ken Thompson
Traditional C	1972	Dennis Ritchie
K & R C	1978	Kernighan & Dennis Ritchie
ANSI C	1989	ANSI Committee
ANSI/ISO C	1990	ISO Committee
C99	1999	Standardization Committee

Features of C language

1) Machine Independent or Portable
2) Mid-level programming language
3) structured programming language
4) Rich Library
5) Memory Management
6) Fast Speed
7) Modular
8) Pointers

By: Dinesh Khadka

Advantages

- Produces portable program
- No limitation for programming (system and application programming)
- C supports dynamic memory management.
- It is fast and efficient
- It is easy for debugging, testing and maintaining.
- Its compiler is easily available
- It has ability extends itself. User can add own functions to the C library.

By: Dinesh Khadka

Disadvantages

- There is no runtime checking in C language, as the program extends it is very difficult to debug.
- There is no strict type checking.
- It is case sensitive
- It may be compile time overhead due to the misplacing and excessive use of pointer.
- It do not support modern programming approach like OOP.

By: Dinesh Khadka

First C program

```
/*First program in C
to display line of text */

#include<stdio.h>

int  main()
{
   printf("welcome to c progtramming");
   return(0);
}
```

By: Dinesh Khadka

First C program …

/*First program in C
to display line of text */

- is a comment.
- Comments start with /* and end with */
- You insert comments to document programs and improve program readability.
- Comments do not cause the computer to perform any action when the program is run.
- Comments are ignored by the C compiler and do not cause any machine-language object code to be generated.
- Comments also help other people read and understand your program.

By: Dinesh Khadka

#include<stdio.h>

- Is a directive to C preprocessor
- The line begins with # is processed by preprocessor before the program is compiled.
- tells the preprocessor to include the contents of the standard input/output header (<stdio.h>) in the program.
- then we can use the function defined in header file stdio.h in program. (printf, scanf)

void main()

- Is a main function and is part of every C program.
- Execution of C program begins from main function .
- Here { indicate the beginning of main program and } indicate the end of main program.

By: Dinesh Khadka

- In the main function the line

printf("welcome to c progtramming");

- instructs the computer to perform an action, namely to print on the screen the string of characters marked by the quotation marks.
- The entire line, including printf, its argument within the parentheses and the semicolon (;), is called a statement.
- Every statement must end with a semicolon (also known as the statement terminator).
- When the preceding printf statement is executed, it prints the message Welcome to c programming on the screen.
- The characters normally print exactly as they appear between the double quotes in the printf statement.

By: Dinesh Khadka

Basic structure of C program

Documentation Section
Link Section
Definition section
Global declaration section

Main ( ) Function section
{
Declaration part
Executable part
}

Sub program section Function1 { } Function2 { } … Function n { }

Fig: Basic structure of C program

By: Dinesh Khadka

- **Documentation section:**
  - Consists of set of comment lines giving the information about program title, author, purpose of program etc.
  - Starts with /* and end with */
- **Link section:** provides the instructions to compiler to link the header files in program.
- **Definition section**: use to define symbolic constants.
- **Global declaration section**: use to declare the global variables as well as used defined functions. Variables declare here will be accessed from any part of program (from all functions).
- **Main( ) function section:** every c from must have a main function. It contains declaration and executables part between { and }.
- **Sub-program section:** contain the definition of user defined functions.

By: Dinesh Khadka

Example

```
/*program to calculate are of circle using function */
#include<stdio.h>
#include<conio.h>
#define PI 3.1428
 float area(float );
void main( )
{
    float a, r;
    printf("Enter radius of circle: ");
    scanf("%f", &r);
    a = area(r);
    printf("Area of circle = %f",a);
    getch();
}

float area(float  r)
{
    float ar;
    ar = PI*r*r;
    retrun(ar);
}
```

Documentation section

Link section

Definition section

Global declaration section

Main function section

Sub program section

By: Dinesh Khadka

Executing a C program

Executing a program written in C involves series of steps. They are

1. Creating the program
   - Once the computer is ready to receive program, enter the program into file.
   - The file is created into text editor such notepad, vi etc.
   - Program can be corrected under editor.
   - Note: file should be save with extension **.c**
   - The program entered into file is called **source program.**
2. Compiling the program
   - After creating/editing the program, it should be compiled.
   - If no syntax error compiler will generate the object program
   - If any error, the edit the source program and re-compile it.
3. Linking the program
   - Linker link the object program with necessary system library and generate the executable file.
4. Executing the program
   - The last step is to run the executable program.
   - Input the necessary data, if wrong data –re-run it and reenter the data.
   - If any logical error, edit source program and repeat step 2 , 3, and 4.
   - If no data and logical error, it produce desired result.

By: Dinesh Khadka

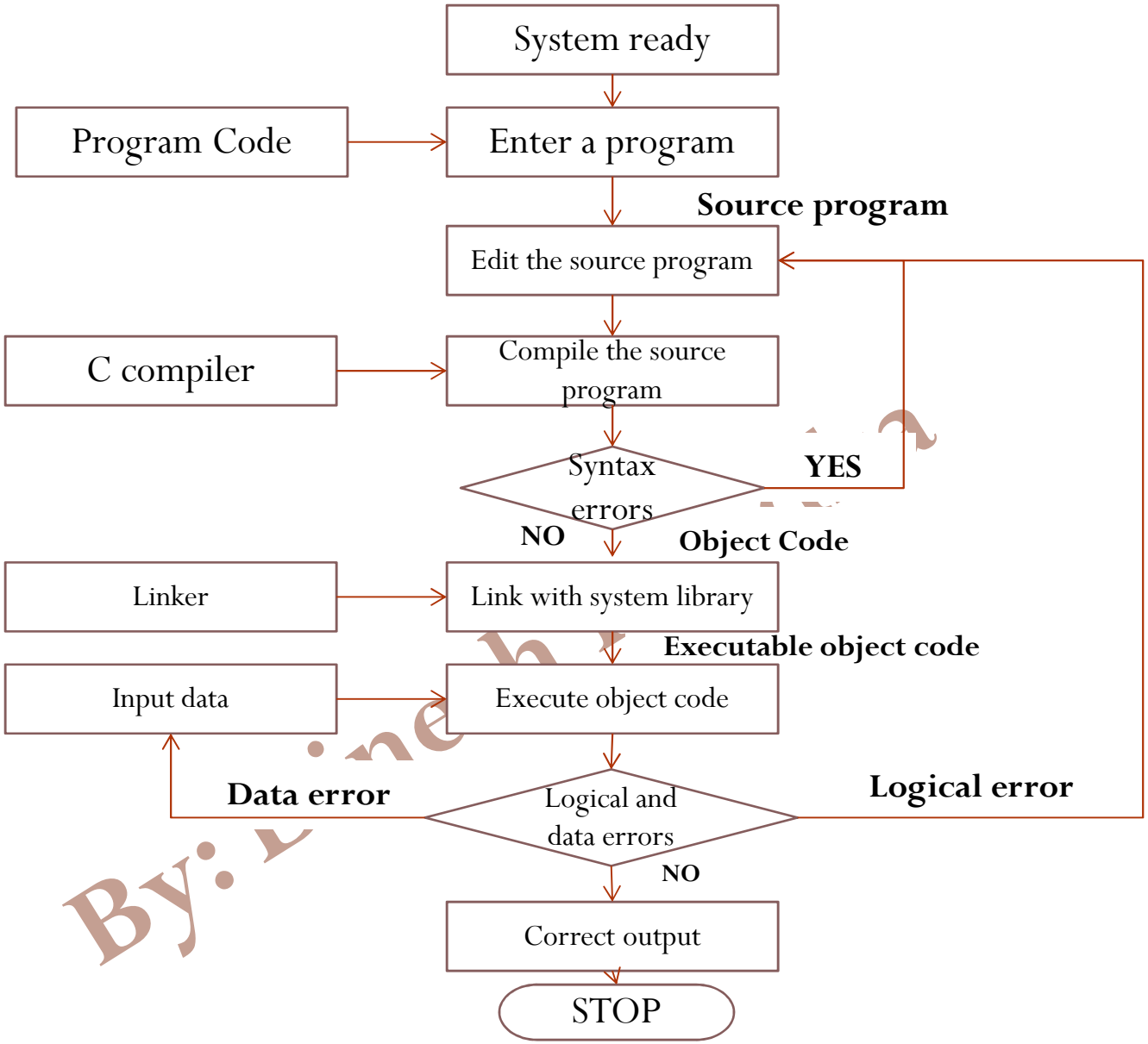


Fig: compiling and running of a C program

The C preprocessor

- Preprocessor is a part of compiler that perform the task before actual compilation.
- Preprocessor accept instruction from preprocessor directives and perform its task before actual compilation of program.
- Preprocessor directives begin with # symbol and do not require semicolon at the end.
- Some commonly used preprocessor directive are:
  - #include - used to specify the header file to be included.
  - #define – to define symbolic constant and macro substitution.
  - #undef - undefine a macro.

Example:

```
#include<stdio.h>
#define TRUE 1
#define PI 3.14
#define MESSAGE "Good Luck"
#undef PI
```

By: Dinesh Khadka

Header file

- Also called include file.
- With extension .h
- contains C function declarations and macro definitions to be shared between several source files.
- There are two types of header files:
  - the files that the programmer writes and
  - the files that comes with your compiler.
- We can request to use a header file in your program by including it with the C preprocessing directive **#include** and it has two form

  #include<header file> -- for system header file and searches standard system directories.

  #include "header file" - for programmer written header file and searches in current directory or specified path
- Example : #include<stdio.h>

By: Dinesh Khadka

Some important header files

Header Files	Meaning
#include<stdio.h>	It is standard input output header file. It contains the definition of every input and output function such as scanf( ) and printf( ) function. It is compulsory in every program.
#include<conio.h>	It stands for Console input output header file. It is a header file which is included in the program that use clrscr( ), getch( ) etc function. It is optional, not necessarily in every C program.
#include<string.h>	This is included for using string handling functions like as strlen(), strcat( ), strcpy( ) etc.
#include<math.h>	This is used to handle mathematical functions like as sqrt(), pow( ), sin( ), cos( ) etc.
#include<graphic.g>	This is used to handle the graphic functions like as line( ), circle( ) etc.
#include<stdlib.h>	Stands for standard library header file . This is used exit(), malloc(), calloc () etc.
#include<ctype.h>	Used for character processing such as for functions: isdigit( ), isalpha( ), toupper( ), tolower( ) etc.

Library function

- Also called built in functions. (distributed with C compiler in executable format)
- Designed for common operation.
- They are declared in header file. So corresponding header file must be included in program.
- Example of library functions are : printf(), pow(), getch(), strrev() etc.
- Using library functions – programming will be easy and quick.
- Some of commonly used library function in math.h with uses are

fabs(double x)	Return the absolute value of x
ceil(double x)	Return a smallest integer value larger or equal to x
floor(double x)	Return a largest integer value smaller or equal to x
sqrt(double x)	Return square root of x
pow(double x, double y)	Return x raised to power y

Dinesh Khadka

Example

```
/* program to that demonstrate
use of pow() functions */
#include<stdio.h>
#include<conio.h>
int main()
{
   int a , b, p;
   a=2, b=11;
    p=pow(a,b);
    printf(" %d raised to power %d = %d",a,b,p);
    return 0;
}
```

By: Dinesh Khadka